UNIVERSIDADE FEDERAL DE SERGIPE

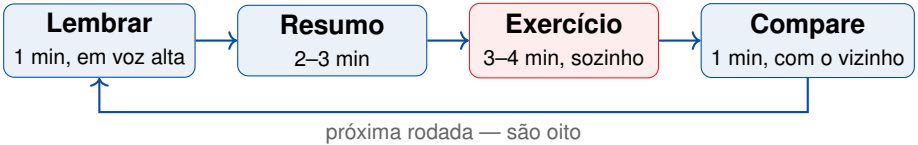
# Exercícios de revisão

## Recursividade, listas, pilhas e filas em oito rodadas

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
andreyoshiaki@dcomp.ufs.br

# Como vai funcionar hoje




## Combinado

- **Tudo no papel, sem computador**: numere uma folha de **R1** a **R8**, responda lendo o código e **desenhe** nós, setas e pilhas — *antes* de conversar com o vizinho.
- O gabarito vem todo junto no final, e cada um confere a própria folha.
- Errou uma rodada? O último slide diz qual aula revisar.

# O caminho que percorremos

da pilha de execução . . .

| | | | |
|---|---|---|---|
| **R1** | Recursão: caso base, ida e volta | **R5** | Lista duplamente encadeada |
| **R2** | Recursão e memória: pilha, cauda e heap | **R6** | Pilhas: vetor e lista |
| **R3** | Lista encadeada: o nó, percorrer e liberar | **R7** | Filas: circular e lista |
| **R4** | Lista: buscar, inserir e remover | **R8** | Síntese: qual estrutura, a que custo? |

. . . às pilhas e filas que nós mesmos construímos

# R1 — Recursão

# Lembrar: recursão

**Antes do resumo: responda em voz alta (1 min)**

- Quais são as duas peças de toda função recursiva?
- O que a pilha de execução guarda a cada chamada?
- Um `printf` *depois* da chamada recursiva roda na ida ou na volta?

Rodada 1 — aulas de origem: recursividade: conceitos e pilha de execução

# Resumo: caso base, passo e a pilha de execução

## Duas peças

**Caso base**: resolve sem chamar a si mesma. **Passo recursivo**: chama a si mesma com um problema *menor*, que caminha para o caso base.

## Pilha de execução

Cada chamada empilha um **registro de ativação** (parâmetros, locais, endereço de retorno); cada `return` desempilha o do topo.

## Ida e volta

O que vem *antes* da chamada acontece na ida (de fora para dentro); o que vem *depois*, na volta, em ordem inversa.

## Recursão múltipla

`fib(n-1) + fib(n-2)` gera uma *árvore* de chamadas, mas a pilha só guarda um galho: altura $\approx n$.

Fonte: Feofiloff (2009), cap. 2 [5]; Bryant & O'Hallaron (2015), §3.7.1 [2]

# Exercício R1 — 4 min, no papel

```c
void f(int n) {
    if (n <= 0)
        return;
    printf("%d ", n);
    f(n - 2);
    printf("%d ", n);
}

int g(int n) {
    if (n < 2)
        return 1;
    return g(n - 1) + g(n - 2);
}
```

## Responda na folha

(a) O que `f(5)` imprime?

(b) Quanto vale `g(4)`? Quantas chamadas a `g` acontecem, contando a primeira?

(c) Durante `g(4)`, quantos registros de `g` ficam na pilha, **no máximo**, ao mesmo tempo?

Dica: em (b) e (c), desenhe a árvore de chamadas.

# R2 — Recursão e memória

# Lembrar: recursão e memória

**Antes do resumo: responda em voz alta (1 min)**

- Por que uma recursão muito profunda termina em *segmentation fault*?
- O que é uma chamada de cauda?
- O que acontece com as variáveis locais de uma função quando ela retorna?

Rodada 2 — aulas de origem: recursividade e efeitos na alocação de memória

# Resumo: o que a recursão gasta, e onde

### Limite da pilha

A pilha tem tamanho fixo (em geral 8 MB, `ulimit -s`). Profundidade $\times$ tamanho do registro passou disso: **estouro de pilha**. O heap é outra região e aguenta muito mais.

### Recursão de cauda

A chamada é a *última* coisa que a função faz: nada fica pendente, e com `-O2` o compilador pode trocá-la por um laço. Sem otimização, não conte com isso.

### Local morre com o `return`

O registro é desempilhado; devolver `&local` entrega um endereço que já não é de ninguém.

### `malloc` em cada chamada

Quem recebe o bloco é dono dele e tem que liberá-lo — na volta, quando já não serve.

Fonte: Kerrisk (2010), §36.3 [7]; Abelson et al. (1996), §1.2.1 [1]; ISO/IEC 9899:2018, §6.2.4 [6]

# Exercício R2 — 4 min, no papel

```c
long conta(int n) {                 /* A */
    if (n == 0) return 0;
    return 1 + conta(n - 1);
}
long conta_acum(int n, long acc) { /* B */
    if (n == 0) return acc;
    return conta_acum(n - 1, acc + 1);
}
int *dobro(int x) {                 /* C */
    int r = 2 * x;
    return &r;
}
```

**Responda na folha**

(a) Qual de **A** e **B** é recursão de cauda? O que fica pendente nos registros da outra?

(b) Pilha de 8 MB, 48 bytes por registro, sem otimização: `conta(100000)` termina? E `conta(1000000)`?

(c) O que há de errado em **C**? Corrija sem mudar o que ela calcula.

# R3 — Lista encadeada

# Lembrar: lista encadeada

**Antes do resumo: responda em voz alta (1 min)**

- O que cada nó guarda além do valor?
- Como se sabe que a lista acabou? E que ela está vazia?
- Inserir no início de um vetor custa quanto? E no de uma lista?

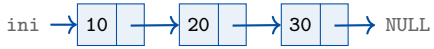
Rodada 3 — aulas de origem: introdução a listas encadeadas

# Resumo: o nó, percorrer, inserir no início, liberar

```c
typedef struct no {
    int valor;
    struct no *prox; /* ou NULL */
} No;

novo->prox = ini;  /* 1o */
ini = novo;        /* 2o */

while (ini != NULL) {
    No *prox = ini->prox; /* antes! */
    free(ini);
    ini = prox;
}
```




- Estrutura **auto-referenciada**: o nó aponta para outro do mesmo tipo; `NULL` marca o fim, `ini == NULL` é a lista vazia.
- Inserir no início é $O(1)$, mas inverte a ordem de entrada.
- Ao liberar, guarde `prox` *antes* do `free`.

Fonte: King (2008), §17.5 [8]; Feofiloff (2009), cap. 4 [5]

# Exercício R3 — 3 min, no papel

```
1  /* No e insere_inicio como no resumo */
2  No *lista = NULL;
3  int entrada[] = {5, 8, 2};
4  for (int i = 0; i < 3; i++)
5      lista = insere_inicio(lista,
6                            entrada[i]);
7  imprime(lista);
8
9  for (No *p = lista; p != NULL;
10      p = p->prox)
11     free(p);
```

**Responda na folha**

(a) Desenhe a lista e escreva o que `imprime` mostra.

(b) Quanto vale `sizeof(No)` numa máquina de 64 bits? Quantos bytes são de padding?

(c) O laço das linhas 9–11 tem um erro. Qual? Corrija.

`int` 4, ponteiro 8.

# R4 — Buscar, inserir, remover

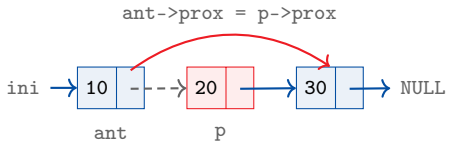
# Lembrar: buscar, inserir e remover

**Antes do resumo: responda em voz alta (1 min)**

- Para inserir ou remover no meio, em que nó o laço precisa parar?
- Que nó é o caso especial da remoção, e por quê?
- Em `p != NULL && p->valor != x`, a ordem importa?

Rodada 4 — aulas de origem: lista encadeada: inserção, remoção e busca

# Resumo: o anterior guia, o primeiro atrapalha




- **Busca**: `while (p != NULL && p->valor != x)` — o `&&` só avalia a direita se a esquerda for verdadeira.
- **Remover**: desviar a seta do *anterior*, depois `free`.
- O **primeiro nó** não tem anterior: quem muda é `ini`, e a função devolve o novo início.
- Com `No **pp` (o endereço da seta a mudar), o caso especial some.

Fonte: Celes et al. (2004), cap. 10 [3]; King (2008), §17.5–17.6 [8]

# Exercício R4 — 4 min, no papel

```
1  No *remove_valor(No *ini, int x)
2  {
3      No *ant = NULL, *p = ini;
4      while (p->valor != x && p != NULL) {
5          ant = p;
6          p = p->prox;
7      }
8      ant->prox = p->prox;
9      free(p);
10     return ini;
11 }
```

## Responda na folha

A lista é `10 -> 20 -> 30 -> NULL`.

(a) `remove_valor(lista, 20)` funciona?

(b) E `remove_valor(lista, 10)`? Em que linha, e por quê?

(c) E `remove_valor(lista, 99)`? Em que linha, e por quê?

(d) Reescreva a função corrigindo os dois casos.

# R5 — Lista duplamente encadeada

# Lembrar: lista duplamente encadeada

**Antes do resumo: responda em voz alta (1 min)**

- O que a lista dupla ganha da simples? E o que ela custa?
- Que regra liga `ant` e `prox` de dois nós vizinhos?
- Por que a lista dupla guarda `ini` *e* `fim`?

Rodada 5 — aulas de origem: lista duplamente encadeada

# Resumo: duas setas por nó, dois extremos por lista




- A regra: `p->prox->ant == p` e `p->ant->prox == p` — e ninguém a confere por você.
- Inserir mexe em **quatro** ponteiros; nos extremos, um deles é `ini` ou `fim`.
- Remover *dado o nó* é $O(1)$: os vizinhos estão nele.
- Dois `if`: sem anterior, muda `ini`; sem seguinte, muda `fim`.
- O nó sentinela dá vizinhos a todo mundo e some com os `if`.

Fonte: Cormen et al. (2009), §10.2 [4]; Knuth (1997), §2.2.5 [9]

# Exercício R5 — 4 min, no papel

```
1  /* encaixa x logo depois de p */
2  novo->ant  = p;
3  novo->prox = p->prox;
4  p->prox->ant = novo;
5  p->prox = novo;
6
7  void remove_no(Lista *l, No *p) {
8      if (p->ant == NULL)
9          l->ini = p->prox;
10     else
11         p->ant->prox = p->prox;
12     if (p->prox != NULL)
13         p->prox->ant = p->ant;
14     free(p);
15 }
```

## Responda na folha

Lista `10 <-> 20 <-> 30`, com `ini` e `fim`.

(a) Encaixe 25 depois do 20 (linhas 2–5). Desenhe. Por que a linha 4 tem de vir *antes* da 5?

(b) `remove_no` está certa para o 20? E para o **último** nó? O que dá errado, e quando aparece?

(c) Corrija `remove_no`.

# R6 — Pilhas

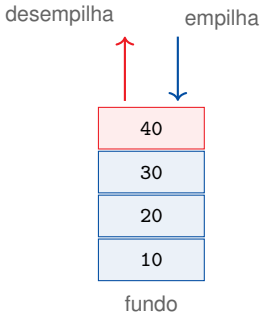
# Lembrar: pilhas

**Antes do resumo: responda em voz alta (1 min)**

- Numa pilha, quem sai primeiro?
- Num vetor, onde fica o topo? E numa lista?
- Quando o vetor enche, por que dobrar, e não somar uma posição?

Rodada 6 — aulas de origem: pilhas: conceito e implementação

# Resumo: o último a entrar é o primeiro a sair




- **Pilha** (LIFO): `empilha`, `desempilha`, `topo`, `vazia` — só o topo é visível.
- **Vetor**: topo em `v[n-1]`. Confira vazia antes de tirar e cheia antes de pôr: “certo por sorte” é errado.
- **Dobrar** a capacidade: poucas cópias, $O(1)$ amortizado por operação. Somar uma constante: cópias demais.
- **Lista**: topo no primeiro nó; insere e remove no início, $O(1)$, sem nunca encher.
- **Balanceamento**: abre empilha; fecha desempilha e tem de combinar; no fim, a pilha tem de estar vazia.

Fonte: Celes et al. (2004), cap. 11 [3]; Tenenbaum et al. (1995), cap. 2 [10]

# Exercício R6 — 4 min, no papel

```c
Pilha *p = pilha_cria();
for (int i = 1; i <= 5; i++)
    empilha(p, 10 * i);
int a = desempilha(p);
int b = desempilha(p);
empilha(p, a - b);
printf("%d\n", topo(p));
while (!vazia(p))
    printf("%d ", desempilha(p));
```

## Responda na folha

(a) O que o programa ao lado imprime?

(b) Para cada entrada, a verificação de balanceamento diz sim ou não? Se não, em que caractere ela percebe, e por quê?
`{[()()]}` `[(])` `{}(`

(c) A pilha começa com capacidade 4. Para empilhar 100 valores, quantas vezes o vetor cresce se **dobrar**? E se **somar 4**?

# R7 — Filas

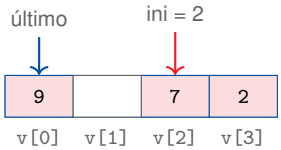
# Lembrar: filas

**Antes do resumo: responda em voz alta (1 min)**

- Numa fila, quem sai primeiro?
- Por que a fila num vetor simples “anda para a direita”?
- Na fila circular, como se distingue cheia de vazia?

Rodada 7 — aulas de origem: filas: conceito e implementação

# Resumo: o primeiro a entrar é o primeiro a sair




frente `v[ini]`; o próximo entra em
`v[(ini + n) % cap]` = `v[(2 + 3) % 4]` = `v[1]`

- **Fila** (FIFO): entra no fim, sai da frente.
- **Circular**: os índices dão a volta com %; guardar `ini` e `n` separa cheia ($n = cap$) de vazia ($n = 0$).
- Para crescer, **desenrole o anel** num vetor novo, frente em `v[0]`.
- **Lista**: sai do primeiro nó, entra depois do último — com `ini` e `fim`, as duas em $O(1)$.

Fonte: Celes et al. (2004), cap. 12 [3]; Cormen et al. (2009), §10.1 [4]

# Exercício R7 — 4 min, no papel

Fila circular com `v`, `ini`, `n` e `cap`; começa vazia, com `cap = 4` e `ini = 0`.

```
1  enfileira(f, 1);
2  enfileira(f, 2);
3  enfileira(f, 3);
4  desenfileira(f);
5  desenfileira(f);
6  enfileira(f, 4);
7  enfileira(f, 5);
```

## Responda na folha

(a) Desenhe `v[0..3]` e dê `ini` e `n` depois da linha 7. Em que posição o próximo valor entra?

(b) Agora `enfileira(f, 6)` e `enfileira(f, 7)`. O 7 não cabe: a fila cresce para `cap = 8` desenrolando o anel. Desenhe `v[0..7]`, `ini` e `n`.

(c) Se o crescimento fosse só `v = realloc(v, 8 * sizeof *v)`, como na pilha, em que ordem os valores sairiam? Por quê?

# R8 — Síntese

# Lembrar: qual estrutura, a que custo?

**Antes do resumo: responda em voz alta (1 min)**

- Pilha e fila dizem *o que* se faz; vetor e lista dizem *como*. Qual é o TAD e qual é a implementação?
- Que operação a lista simples *não* faz bem, mesmo com `fim`?

Rodada 8 — aulas de origem: todas as anteriores

# Exercício R8 — 4 min, no papel

## (a) Pilha, fila, lista simples ou lista dupla?

1. Desfazer (`Ctrl+Z`) num editor de texto.
2. Pedidos de impressão, atendidos na ordem de chegada.
3. Playlist com “anterior”, “próxima” e “remover a atual”.
4. Imprimir os $n$ números lidos na ordem *inversa*, sem saber $n$ antes.

Justifique cada escolha em uma linha.

## (b) Complete com $O(1)$ ou $O(n)$

| | vetor `v[0..n-1]` | simples `ini, fim` | dupla `ini, fim` |
|---|---|---|---|
| inserir no início | | | |
| inserir no fim | | | |
| remover do início | | | |
| remover do fim | | | |

O vetor tem folga (`n < cap`) e não é circular.

# Gabarito

# Hora de conferir

## Como conferir

Marque na folha: **C** (certo), **P** (parcial) ou **E** (errado) em cada rodada. O placar diz o que revisar antes da prova — está no último slide.

## Quer ver rodando?

Cada rodada tem um programa em `codigo/` e uma seção no notebook: a saída tem de bater com o gabarito.

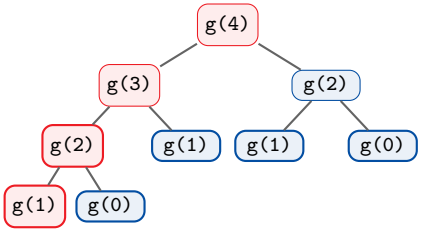
# Gabarito R1: ida, volta e a árvore de `g(4)`

**(a)** `5 3 1 1 3 5`

Na ida, 5, 3, 1; `f(-1)` para no caso base; na volta, 1, 3, 5.

**(b)** `g(4)` $=$ **5**, em **9** chamadas (os nós da árvore).

**(c) 4**: o galho mais comprido, `g(4)`$\rightarrow$`g(3)`$\rightarrow$`g(2)`$\rightarrow$`g(1)`. Os ramos irmãos só entram depois que o anterior saiu.




Em vermelho, o galho que a pilha guarda no momento mais fundo.

Fonte: Feofiloff (2009), cap. 2 [5]

# Gabarito R2: cauda, limite da pilha e local morta

(a) **B** é de cauda: devolve direto o resultado da chamada. Em **A**, cada registro guarda o "`1 +`" que só roda na volta.

(b) $100\,000 \times 48 = 4{,}8$ MB: **termina**.
$1\,000\,000 \times 48 = 48\,\text{MB} > 8\,\text{MB}$: **estoura a pilha** (*segmentation fault*).

(c) `r` vive no registro de `dobro`, que morre no `return`: o endereço devolvido é de memória que será reaproveitada.

```c
int dobro(int x)        /* o valor */
{                       /* sobrevive */
    return 2 * x;
}
```

```
warning: function returns address of
local variable [-Wreturn-local-addr]
```

Ou `void dobro(int x, int *saida)`, ou `malloc` — e aí quem recebe dá `free`.

Fonte: Abelson et al. (1996), §1.2.1 [1]; ISO/IEC 9899:2018, §6.2.4 [6]

# Gabarito R3: a ordem invertida e o `free` apressado

**(a)** Cada valor entra na frente do anterior:

```
2 -> 8 -> 5 -> NULL
```

**(b)** `int` (4) + **4 de padding** + ponteiro (8): `sizeof(No)` = **16**. O ponteiro precisa começar num múltiplo de 8.

**(c)** O incremento `p = p->prox` roda *depois* do `free(p)`: lê um nó já devolvido (`heap-use-after-free` no `-fsanitize=address`). Guarde o caminho antes:

```
while (lista != NULL) {
    No *prox = lista->prox;
    free(lista);
    lista = prox;
}
```

Fonte: King (2008), §17.5 [8]; Bryant & O'Hallaron (2015), §3.9.3 [2]

# Gabarito R4: o meio funciona, as pontas não

(a) **Sim**: para em `p` $= 20$, `ant` $= 10$, e o 10 passa a apontar para o 30.

(b) **Quebra na linha 8**: o 10 é o primeiro, o laço nem roda e `ant` continua `NULL` — `ant->prox` é *segfault*. Quem tinha de mudar era `ini`.

(c) **Quebra na linha 4**: `p` chega a `NULL` e `p->valor` é lido *antes* do teste `p != NULL`. Inverta o `&&`; não achou, nada muda.

```
No *remove_valor(No *ini, int x) {
    No *ant = NULL, *p = ini;
    while (p != NULL && p->valor != x) {
        ant = p;
        p = p->prox;
    }
    if (p == NULL) return ini; /* nao achou
     */
    if (ant == NULL)      /* era o 1o */
        ini = p->prox;
    else
        ant->prox = p->prox;
    free(p);
    return ini;
}
```

Fonte: Celes et al. (2004), cap. 10 [3]

# Gabarito R5: quatro ponteiros e o `fim` esquecido

(a) `10 <-> 20 <-> 25 <-> 30`. A linha 4 usa `p->prox` para chegar ao 30; se a 5 vier antes, `p->prox` já é `novo`, e a linha 4 faz `novo->ant = novo` — o 30 fica apontando para o 20.

(b) Para o 20, certa. Para o **último**, o `if` da linha 12 é falso e ninguém atualiza `l->fim`: ele continua apontando para o nó liberado. O erro só aparece *depois*, quando alguém percorre de trás para frente — `heap-use-after-free` no `imprime`.

(c) Simétrico ao primeiro `if`:

```
void remove_no(Lista *l, No *p) {
    if (p->ant == NULL)
        l->ini = p->prox;
    else
        p->ant->prox = p->prox;
    if (p->prox == NULL)
        l->fim = p->ant;
    else
        p->prox->ant = p->ant;
    free(p);
}
```

Fonte: Cormen et al. (2009), §10.2 [4]

# Gabarito R6: pilhas

**(a)** Sai 50, depois 40; entra $50 - 40 = 10$:

```
10
10 30 20 10
```

O 10 do topo é o *calculado*; o do fundo, o primeiro empilhado.

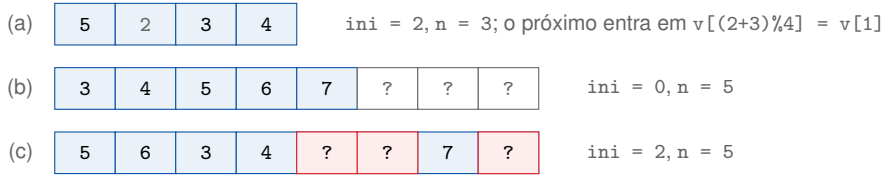
**(c)** Dobrando:
$4 \to 8 \to 16 \to 32 \to 64 \to 128$, **5** vezes.
Somando 4: $4 + 4k \geq 100$, **24** vezes — cada uma copia tudo o que já havia.

**(b)**

- `{[()()]}`: **sim**.
- `[(])`: **não**, no `]`: o topo é `(`, e `(` não fecha com `]`.
- `{}(`: **não**, só **no fim** da entrada: sobrou `(` na pilha. Nenhum fechamento estava errado.

Fonte: Celes et al. (2004), cap. 11 [3]

# Gabarito R7: o anel, desenrolado ou não




- (a) Saíram 1 e 2; o 5 deu a volta e caiu em `v[0]`. O 2 em `v[1]` é resto: não está entre os `n`.
- (b) O 6 vai para `v[1]` e enche a fila; o 7 provoca o crescimento: o anel é copiado em ordem a partir da frente.
- (c) Com `realloc`, 5 e 6 continuam em `v[0..1]`, mas depois de `v[3]` agora vem `v[4]`, e não `v[0]`. O 7 entra em `v[(2+4)%8] = v[6]`, e saem **3, 4, lixo, lixo, 7** — 5 e 6 se perderam.

Fonte: Celes et al. (2004), cap. 12 [3]

# Gabarito R8: estrutura e custo

**(a)**

1. **Pilha**: desfaz-se a última ação primeiro.
2. **Fila**: quem chegou antes imprime antes.
3. **Lista dupla**: anda nos dois sentidos e remove o nó atual sem procurar o anterior.
4. **Pilha** (numa lista, que não enche): o último lido sai primeiro. Uma recursão também resolve — a pilha é a de execução.

**(b)**

| | vetor | simples | dupla |
|---|---|---|---|
| inserir no início | $O(n)$ | $O(1)$ | $O(1)$ |
| inserir no fim | $O(1)$ | $O(1)$ | $O(1)$ |
| remover do início | $O(n)$ | $O(1)$ | $O(1)$ |
| remover do fim | $O(1)$ | $O(n)$ | $O(1)$ |

No vetor, o início desloca todo mundo. Na simples, remover o último exige achar o *penúltimo*. Por isso a fila em lista sai do início e entra no fim — e a circular tira o $O(n)$ do vetor.

Fonte: Cormen et al. (2009), §10.1–10.2 [4]

# Seu placar: onde revisar

| Errou | Revise a aula — e refaça as tarefas e o desafio do fim dela |
|---|---|
| R1 | Recursividade: conceitos e pilha de execução |
| R2 | Recursividade e efeitos na alocação de memória |
| R3 | Introdução a listas encadeadas |
| R4 | Lista encadeada: inserção, remoção e busca |
| R5 | Lista duplamente encadeada |
| R6 | Pilhas: conceito e implementação |
| R7 | Filas: conceito e implementação |
| R8 | Os “Quanto custa” das aulas de listas, pilhas e filas |

**Para casa**

Rode o notebook: cada exercício está lá, pronto para executar e conferir a sua previsão.

# Referências I


- Harold Abelson, Gerald Jay Sussman, and Julie Sussman.
  *Structure and Interpretation of Computer Programs*.
  MIT Press, Cambridge, MA, 2 edition, 1996.
- Randal E. Bryant and David R. O'Hallaron.
  *Computer Systems: A Programmer's Perspective*.
  Pearson, Boston, 3 edition, 2015.
- Waldemar Celes, Renato Cerqueira, and José Lucas Rangel.
  *Introdução a estruturas de dados: com técnicas de programação em C*.
  Elsevier, Rio de Janeiro, 2004.

# Referências II



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
*Introduction to Algorithms*.
MIT Press, Cambridge, MA, 3 edition, 2009.


Paulo Feofiloff.
*Algoritmos em linguagem C*.
Elsevier, Rio de Janeiro, 2009.


ISO/IEC.
Iso/iec 9899:2018 — information technology: Programming languages: C.
Technical report, International Organization for Standardization, 2018.

# Referências III



Michael Kerrisk.
*The Linux Programming Interface*.
No Starch Press, San Francisco, 2010.


K. N. King.
*C Programming: A Modern Approach*.
W. W. Norton, New York, 2 edition, 2008.


Donald E. Knuth.
*The Art of Computer Programming, Volume 1: Fundamental Algorithms*.
Addison-Wesley, Reading, MA, 3 edition, 1997.

# Referências IV


Aaron M. Tenenbaum, Yedidyah Langsam, and Moshe J. Augenstein.
*Estruturas de dados usando C*.
Pearson Makron Books, São Paulo, 1995.